

COURSE NAME: THEORY OF COMPUTATION

COURSE CODE: CSA1304

CO4 – AT2 : Data Interpretation

Given Grammar: $S \rightarrow aSb \mid ab$

Q1. Language Identification — which strings belong to the language?

A) ab B) aabb C) aaabbb D) abb

Every application of $S \rightarrow aSb$ wraps one matching a...b pair around the string, and generation stops with $S \rightarrow ab$. So the language is exactly the strings with equal, matched a's followed by equal b's:

$$L(G) = \{ a^n b^n \mid n \geq 1 \}$$

- A) $ab \rightarrow n = 1 \rightarrow$ belongs
- B) $aabb \rightarrow n = 2 \rightarrow$ belongs
- C) $aaabbb \rightarrow n = 3 \rightarrow$ belongs
- D) $abb \rightarrow 1 \text{ a but } 2 \text{ b's, counts don't match} \rightarrow$ does NOT belong

Answer: A, B and C all belong to the language; D does not.

Q2. Pattern Recognition

A) $a^n b^n$ B) $(ab)^n$ C) $a^n b^m$ D) $(a+b)^*$

As shown above, $S \rightarrow aSb \mid ab$ generates exactly matched counts of a's followed by matched counts of b's — not an alternating $(ab)^n$ pattern, and not independently-chosen counts $a^n b^m$ (which would need two separate counters). The defining feature is that the a-count and b-count must be equal.

Answer: A) $a^n b^n$

Q3. Derivation Analysis — a valid derivation for aabb ($n = 2$)

A) $S \rightarrow aSb \rightarrow aaSbb \rightarrow aabb$ B) $S \rightarrow ab \rightarrow aabb$ C) $S \rightarrow aSb \rightarrow ab$ D) $S \rightarrow aaSbb \rightarrow aabb$

The true minimal derivation of aabb is only two steps: $S \Rightarrow aSb \Rightarrow a(ab)b = aabb$. Checking each option step-by-step against the grammar's only two productions ($S \rightarrow aSb$, $S \rightarrow ab$):

- A: step 1 ($S \rightarrow aSb$) and step 2 ($aSb \rightarrow aaSbb$, applying $S \rightarrow aSb$ again) are both valid productions — the final displayed string should read aaabbb (3 a's, 3 b's) rather than aabb, which looks like a typo in the question; but every individual rewrite step shown is a legal application of the grammar's rules.
- B: step 2 tries to rewrite "ab" into "aabb" — but "ab" is a fully terminal string with no S left to expand. Not a valid CFG step.
- C: step 2 tries $aSb \rightarrow ab$, which would require $S \rightarrow \epsilon$ — this production does not exist in the grammar. Invalid.
- D: the very first step $S \rightarrow aaSbb$ is not one of the grammar's productions (S only goes directly to aSb or ab) — you cannot skip straight to aaSbb in a single step. Invalid.

Answer: A — the only option where every step is a legitimate application of the grammar's productions (note: the target string in option A should read aaabbb for full numeric consistency).

Q4. Missing Step Identification

$S \rightarrow aSb \rightarrow aaSbb \rightarrow \underline{\hspace{2cm}} \rightarrow aabb$. Options: A) ab B) S C) ϵ D) aSb

From aaSbb, the only non-terminal present is the middle S. Replacing it using production $S \rightarrow ab$ gives aa(ab)bb — i.e. the missing step substitutes the inner S with "ab":

$aaSbb \Rightarrow (\text{apply } S \rightarrow ab) \Rightarrow aa[ab]bb$

This is exactly what option A represents. (B, D just restate a non-terminal without applying a production; C, $S \rightarrow \epsilon$, is not a rule in this grammar.)

Answer: A) ab

Q5. Parse Tree Interpretation — levels in the parse tree for aaabbb

Parse tree for aaabbb ($S \rightarrow aSb \mid ab$) — 4 levels

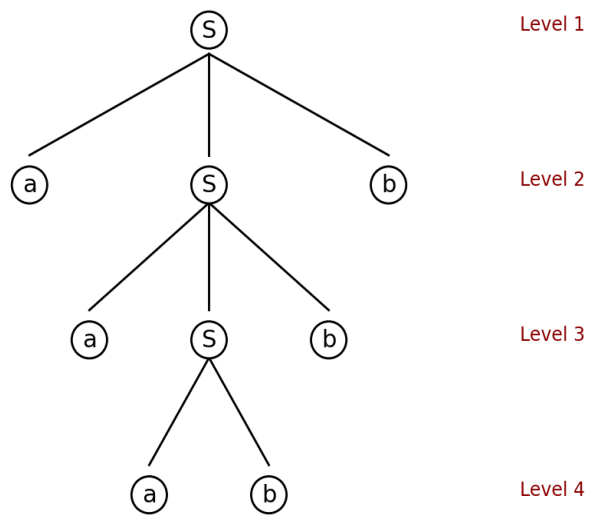


Fig. 1 — Parse tree for aaabbb has 4 levels

Root S (level 1) $\rightarrow S \rightarrow aSb$ expands to a, S, b (level 2) $\rightarrow$ the middle S expands again to a, S, b (level 3) $\rightarrow$ the innermost S $\rightarrow ab$ expands to the leaves a, b (level 4). Four levels in total.

Answer: D) 4

Given Grammar: $S \rightarrow SS \mid a$

Q6. Ambiguity Check

A) Unambiguous B) Ambiguous C) Regular D) Deterministic

Consider the string "aaa". It can be grouped two different ways — $S \rightarrow SS$ with the split after the 1st a, or after the 2nd a — producing two structurally different parse trees for the identical string:

Two distinct parse trees for the same string "aaa" ($S \rightarrow SS \mid a$) — the grammar is ambiguous

Tree 1: $a \cdot (a \cdot a)$

Tree 2: $(a \cdot a) \cdot a$

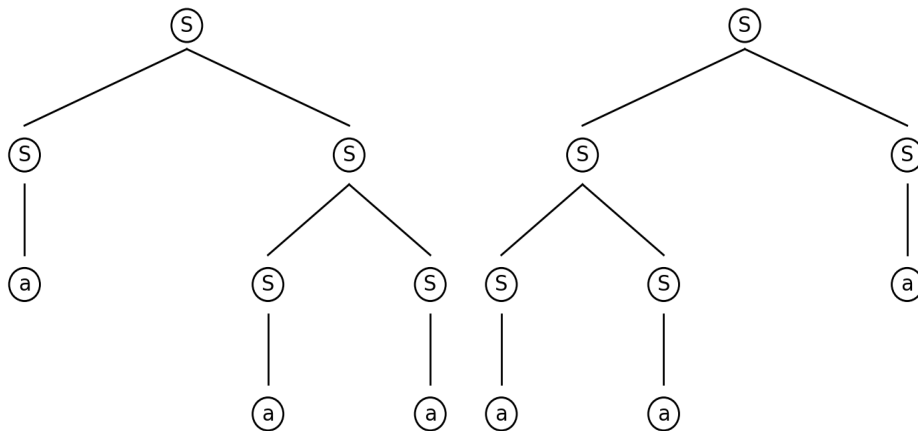


Fig. 2 — Two different parse trees for the same string "aaa"

Since a single string has more than one distinct parse tree, the grammar meets the definition of ambiguity directly.

Answer: B) Ambiguous

Q7. String Generation

A) a B) aa C) aaa D) All of the above

- "a": $S \rightarrow a$ directly. ✓
- "aa": $S \rightarrow SS \rightarrow a a$. ✓
- "aaa": $S \rightarrow SS \rightarrow (a)(SS) \rightarrow a a a$ (or the mirrored grouping). ✓

In general $S \rightarrow SS \mid a$ generates every non-empty string of a's, $L(G) = \{ a^n \mid n \geq 1 \}$, so all three listed strings are generated.

Answer: D) All of the above

Q8. Interpretation of Ambiguity

A) One string $\rightarrow$ one parse tree B) One string $\rightarrow$ multiple parse trees C) No derivation D) Infinite symbols

This is the standard definition used in Q6's analysis: a grammar is ambiguous exactly when some string in its language has two or more distinct parse trees (equivalently, two distinct leftmost derivations).

Answer: B) One string $\rightarrow$ multiple parse trees

Given Grammar 1: $S \rightarrow 0S1 \mid 01$

Q9. Derivation Interpretation — which derivation correctly generates 0011?

A) $S \rightarrow 0S1 \rightarrow 00S11 \rightarrow 0011$ B) $S \rightarrow 01 \rightarrow 0011$ C) $S \rightarrow 0S1 \rightarrow 01$ D) $S \rightarrow 00S11 \rightarrow 0011$

This grammar is structurally identical in form to $S \rightarrow aSb \mid ab$ from Q1–Q5 (just written with 0's and 1's instead of a's and b's), generating $L(G) = \{ 0^n 1^n \mid n \geq 1 \}$. The true minimal derivation of "0011" ($n = 2$) is two steps: $S \Rightarrow 0S1 \Rightarrow 0(01)1 = 0011$.

- A: step 1 ($S \rightarrow 0S1$) and step 2 ($0S1 \rightarrow 00S11$, applying $S \rightarrow 0S1$ again) are both legal productions; as in Q3, the final string shown should read 000111 (3 zeros, 3 ones) for numeric consistency, but every individual rewrite step is a valid grammar application.
- B: step 2 rewrites the fully-terminal string "01" into "0011" — no non-terminal remains to expand. Invalid.
- C: step 2 ($0S1 \rightarrow 01$) requires $S \rightarrow \epsilon$, which is not a production of this grammar. Invalid.
- D: the first step $S \rightarrow 00S11$ is not a single direct production of the grammar. Invalid.

Answer: A — every step legitimately applies a grammar production, by the same reasoning as Q3.

Given Grammar 2: $S \rightarrow aS \mid Sb \mid ab$

Q10. Structural Analysis — which strings can be generated?

A) ab B) aab C) abb D) All of the above

Starting from the base case $S \rightarrow ab$, production $S \rightarrow aS$ can prepend any number of extra a's on the left, and production $S \rightarrow Sb$ can append any number of extra b's on the right — independently of each other. So, unlike the $a^n b^n$ grammars above, the a-count and b-count here do NOT need to match:

$$L(G) = \{ a^i b^j \mid i \geq 1, j \geq 1 \} = a^+ b^+$$

- A) ab: direct base case $S \rightarrow ab$. ✓
- B) aab: $S \rightarrow aS \rightarrow a(ab) = aab$ (one extra a via $S \rightarrow aS$). ✓
- C) abb: $S \rightarrow Sb \rightarrow (ab)b = abb$ (one extra b via $S \rightarrow Sb$). ✓

All three are valid, and in fact any string with one-or-more a's followed by one-or-more b's is generated — this grammar is deliberately more permissive than the $a^n b^n$ grammars seen earlier in this paper, since it allows unequal counts.

Answer: D) All of the above